Porte logiche e circuiti logici

Marco Cerbella

Lezione 3

1 Porte logiche

Definizione. [Porta logica] Una *porta logica* è un componente elettronico di base che opera su uno o più segnali di ingresso e genera un unico segnale in uscita. È la realizzazione fisica delle funzioni elementari introdotte dall'algebra (AND, OR, NOT).

Oltre alle porte AND, OR e NOT, si usano le porte NAND, NOR, XOR e XNOR.

x_0	x_1	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

La porta NOT ha un solo ingresso e restituisce x'.

1.1 NAND e NOR: operatori funzionalmente completi

La porta NAND realizza $(x_0 \cdot x_1)'$ e la porta NOR realizza $(x_0 + x_1)'$.

Proposizione. NAND e NOR sono *funzionalmente completi*: qualsiasi funzione logica si può realizzare usando esclusivamente porte NAND, oppure esclusivamente porte NOR, invece di AND, OR e NOT.

Esempio (NOT con una NAND)**.** Collegando i due ingressi di una NAND alla stessa variabile x si ottiene, per idempotenza,

$$(x \cdot x)' = (x)' = x'.$$

Quindi la porta NOT si realizza con una NAND.

1.2 XOR e XNOR

Le porte XOR e XNOR corrispondono agli operatori booleani XOR e XNOR (OR e NOR *esclusivi*): realizzano rispettivamente la funzione di *disuguaglianza* e di *uguaglianza* tra due bit. Lo XOR vale 1 se e solo se i due ingressi sono diversi, lo XNOR vale 1 se e solo se sono uguali.

1.3 Porte a n ingressi

Accanto alle porte elementari a due ingressi, in genere si assume di avere a disposizione le stesse porte con un numero arbitrario di ingressi. In una AND a tre ingressi x_0, x_1, x_2 l'uscita y vale

1 se e solo se tutte le variabili in ingresso valgono 1. In una OR a quattro ingressi $x_0, \ldots, x_3$ l'uscita y vale 1 se almeno una variabile in ingresso vale 1.

2 Circuiti logici

Le porte elementari si collegano per realizzare funzioni più complesse. A partire da un'espressione logica e (ricavata, per esempio, dalla tabella di verità di una funzione booleana) si costruisce il circuito che realizza la funzione descritta da e.

Esempio. Il circuito per $x + y'z$ usa una porta NOT su y, una AND tra y' e z (uscita $y'z$) e una OR tra x e $y'z$.

2.1 Ritardi e cammino critico

Ogni porta logica ha un *ritardo*: il tempo tra l'istante in cui si applicano i segnali in ingresso e l'istante in cui il segnale di uscita si stabilizza a un valore costante. Il ritardo dipende dalla realtà fisica dei componenti (i transistor).

> **Definizione.** [Cammino critico] Il *cammino critico* (*critical path*) è il cammino dagli ingressi alle uscite che attraversa il maggior numero di porte in un circuito, trascurando i negatori, perché possono essere accorpati alla porta precedente o successiva.

Esempio. In una rete divisa in tre livelli di porte, i cammini critici attraversano tre porte.

2.2 Circuiti da forme canoniche

Le funzioni espresse in prima o in seconda forma canonica corrispondono sempre a circuiti logici *a due livelli*, che usano porte a n ingressi.

- In *prima forma canonica* il primo livello contiene le porte AND che calcolano i mintermini e il secondo livello un'unica porta OR che ne fa la somma.
- In *seconda forma canonica* il primo livello contiene le porte OR che calcolano i maxtermini e il secondo livello un'unica porta AND che ne fa il prodotto.

Esempio. Sia $f(x, y, z)$ la funzione che vale 1 solo per 011, 101, 110:

x	y	z	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

La prima forma canonica è $f = x'yz + xy'z + xyz'$. Il circuito ha tre NOT per le variabili negate, tre AND a tre ingressi (una per mintermine) e una OR a tre ingressi.

3 Circuiti speciali: i sommatori

Con le porte logiche si costruiscono circuiti che eseguono operazioni aritmetiche. Vediamo tre sommatori.

3.1 Half adder

Un *half adder* (HA) ha come ingressi due bit a, b e come uscite due bit: s (la somma) e c (il riporto, *carry*).

a	b	s	c
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

La colonna s è uno XOR, la colonna c è un AND:

$$s = a \oplus b, \qquad c = a \cdot b.$$

Il circuito è quindi formato da una porta XOR e una porta AND con gli stessi ingressi.

3.2 Full adder

Un *full adder* (FA) ha tre ingressi, a, b e c_{in} (bit e riporto in ingresso), e due uscite, s e c_{out} (somma e riporto in uscita).

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Poiché la somma è associativa, $a + b + c_{in} = (a + b) + c_{in}$: si possono usare due half adder, il primo per $a + b$ e il secondo per sommare c_{in} al risultato, e combinare i due riporti con una OR per ottenere c_{out}.

3.3 Sommatore a n bit (ripple-carry)

Un *sommatore a n bit* ha come ingressi $2n$ bit, $a_{n-1} \dots a_0$ e $b_{n-1} \dots b_0$, e come uscite n bit $s_{n-1} \dots s_0$ più il riporto finale c, tali che la sequenza in uscita rappresenti in binario la somma dei due numeri in ingresso. Si realizza collegando n full adder in cascata: il riporto in uscita di ciascuno entra come riporto in ingresso del successivo (da cui il nome *ripple-carry*).

Esempio. Per sommare due numeri a 3 bit $X = x_2x_1x_0$ e $Y = y_2y_1y_0$ servono 3 full adder in cascata. Ad esempio $100 + 110 = 1010$.

Riferimenti. Libro di testo (C. Bolchini et al., *Reti Logiche*, Maggioli): cap. 2, sottosezioni 2.6 (porte logiche) e 2.7 (circuiti logici); cap. 5, sottosezione 5.2.1 (sommatori ripple-carry). Esercizi 2.24, 2.25, 2.26, 2.27.